

Cognitive Fabric: An Executable Formalism for Snapshot-Consistent Adaptive Coalitions of Local Experts

1Xq Mathematical Architecture Specification

Rob Tharpe

July 23, 2026

Abstract

This paper specifies a Cognitive Fabric as a finite, immutable registry of localized selector experts evaluated against an explicit context. A selector is defined as a reversible coordinate chart, a bounded positive-definite precision geometry, and a complete family of candidate scoring models. An exactly optimized bounded coalition combines selector reports through a prior-tempered logarithmic opinion pool. A posterior-alignment certificate separates winning confidence, agreement, independent support, redundancy, dissent, uncertainty, and novelty. Structural proposals are typed, evidence-bearing advisory values and are deliberately excluded from the state transition.

The principal results are: composition reversibility; certified lower, upper, and condition-number bounds for selector precision; local and coalition simplex invariants; vote-mass conservation; exact optimality of the bounded coalition; a tempered log-pool identity and prior-neutrality result; deterministic replay for an explicit clock and seed; per-resolution snapshot consistency; mechanically checked bounded-queue conservation, commit classification, and foreign-source isolation in the abstraction; generation monotonicity; and structural-proposal non-interference. The publication protocol is specified as a TLA+ transition system. TLAPS proves its safety invariant for arbitrary admissible parameters: 747 obligations were proved, followed by a clean Isabelle reconstruction run. TLC supplies a finite-state cross-check. A standard-library Python implementation is presented as one finite-precision realization, together with executable conformance tests. Algebraic claims are stated over exact reals; machine claims are explicitly tolerance-bounded and rely on the stated Python lock semantics and SHA-256 collision-resistance assumption.

Keywords: mixture of experts; formal architecture; snapshot consistency; opinion pool; coalition selection; reversible transforms; immutable generations; auditable adaptive systems.

1 Introduction

Adaptive decision systems often combine routing, expert models, online learning, uncertainty heuristics, and concurrent state publication in one implementation. Such a program can be useful while remaining difficult to reason about: terms such as “selector”, “coalition”, and “posterior” may denote whatever the current code happens to compute, and concurrency claims may depend on accidental runtime behavior.

The Cognitive Fabric takes the opposite direction. It first fixes a mathematical state space and an evaluation relation. The implementation is then judged as a realization of that model. A decision is not only a candidate identifier. It is an immutable certificate containing the source generation and fingerprint, the explicit evaluation context, every evaluated selector vote, the exact bounded coalition, the generalized posterior, alignment diagnostics, and any structural advisories.

The contribution is not a claim that local experts, spline models, reversible maps, probability pooling, or immutable versions are individually new. Mixtures of local experts have a long history [1, 2, 3]; logarithmic pooling belongs to the literature on combining expert distributions [4, 5]; invertible representations are central to normalizing-flow work [11]; and versioned state is foundational in concurrency control [12, 21]. The defensible contribution is their precise composition into an auditable adaptive architecture with cross-layer invariants, an explicit refinement boundary, and executable evidence.

1.1 Contributions

This specification contributes:

1. exact definitions of a selector, selector vote, bounded coalition, operational posterior, posterior alignment, structural proposal, snapshot, decision, outcome, and publication transition;
2. a precision construction whose spectral floor, ceiling, and condition bound hold by construction rather than by an approximate diagnostic;
3. an exact coalition optimizer over the bounded evaluated set, with a canonical tie rule and a maximum of fifteen feasible coalitions under the default preselection bound;
4. a prior-tempered logarithmic opinion pool with nonnegative prior residual weight;
5. a pure evaluation relation once logical time and exploration seed are explicit;
6. a TLA+ snapshot-publication protocol with a TLAPS-checked inductive safety theorem, plus a restricted-history implementation linearization-point argument and in-process exception safety for queued evidence; and
7. a formal-to-code traceability map and an executable conformance suite.

1.2 Scope and non-claims

The term *posterior* in this paper means the normalized distribution defined by the operational model. It is a generalized posterior, not an automatic claim of calibration or equivalence to a generative Bayesian model. The quantity named expected information gain is an entropy-weighted heuristic, not mutual information. Structural proposals are not self-modification: they are advisory records with no direct transition semantics. Learning is not algebraically reversible. The proved reversibility is coordinate reversibility; state rollback requires retention of an earlier immutable snapshot. Property tests are evidence of conformance, not machine-checked proofs. The TLA+/TLAPS result is a machine-checked theorem about the abstract publication module; it is not yet a statement-level refinement proof for the Python interpreter, nor a generic linearizability proof against an atomic reference object. It does not mechanize the selector reversibility, coalition, posterior, or structural-proposal theorems, which remain mathematical proofs with conformance tests.

2 Architectural overview

Figure 1 shows the separation of immutable state, evaluation, delayed evidence, and publication.

The architecture has three boundaries:

Evaluation boundary.

A resolution captures exactly one snapshot reference and returns a complete decision certificate.

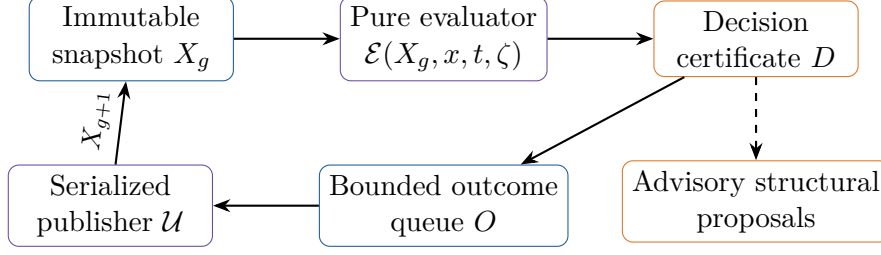


Figure 1: The evaluator consumes one immutable generation. Structural proposals branch from the decision but cannot mutate the snapshot. Only the serialized publisher constructs and swaps a successor generation.

Evidence boundary.

Outcomes are bound to the exact source snapshot identity. At publication, queued stale or foreign-branch outcomes are classified as rejected; before publication, bounded overflow may instead count them as dropped.

Publication boundary.

A valid evidence batch is folded into a new immutable generation and published atomically at the in-process API boundary under the two assumed locks. This is neither crash atomicity nor durable atomicity. The prior snapshot is not mutated.

3 Preliminaries and admissible snapshots

Let $d \in \mathbb{N}_{>0}$ be the common dimension. For $a \leq b$, define

$$[x]_a^b = \min(b, \max(a, x)).$$

Let $\epsilon = 10^{-12}$ and $\text{saf}(x) = [x]_\epsilon^1$. All sets below are finite. The implementation represents semantic registries in ascending identifier order.

Definition 3.1 (Snapshot). An immutable Cognitive Fabric snapshot is

$$X = (g, \mathcal{T}, \mathcal{C}, \mathcal{S}, \mathcal{E}, \Lambda_0, \tau),$$

where $g \in \mathbb{N}$ is a generation, $\mathcal{T}$ is a transform registry, $\mathcal{C}$ a candidate registry, $\mathcal{S}$ a selector registry, $\mathcal{E}$ a directed selector-edge registry, Λ_0 a fallback precision model, and $\tau \in \mathbb{N}$ the creation time.

Definition 3.2 (Well-formed snapshot). A snapshot is well formed when:

1. transform, candidate, selector, and directed-edge keys are unique;
2. at least one candidate is enabled and at least one selector is routable;
3. every transform, selector center, precision, and spline model has dimension d ;
4. every transform reference and edge endpoint resolves;
5. every selector contains exactly one model for every candidate;
6. all semantic numbers are finite and satisfy their declared ranges;
7. $0 \leq \text{successes} \leq \text{observations}$, reward means lie in $[0, 1]$, and latency means are nonnegative; and

8. every registered transform belongs to a certified immutable transform variant.

The function `validate_snapshot` enforces these conditions in the reference realization. Constructor inputs are canonicalized into tuples before publication, preventing mutation of caller-owned lists from changing an existing snapshot.

Definition 3.3 (Canonical snapshot identity). Let $\text{enc}(X)$ be a type-tagged, field-complete canonical JSON encoding in which registries are identifier-sorted and finite floating-point values are encoded by their hexadecimal representation. The implementation identity is

$$h(X) = \text{SHA256}(\text{CF-1XQ-1.0} \parallel \text{enc}(X)).$$

In the mathematical model, h is treated as an injective content identifier. In the realization, direct foreign-evidence exclusion depends on SHA-256 collision resistance.

4 Reversible coordinate charts

Definition 4.1 (Certified transform). A transform θ is a tuple

$$\theta = (id_\theta, F_\theta, G_\theta, \kappa_\theta)$$

where $F_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is forward evaluation, $G_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is its inverse, and $\kappa_\theta \in \mathbb{R}_{\geq 0}$ is a complexity cost. Exact admissibility requires

$$G_\theta \circ F_\theta = I, \quad F_\theta \circ G_\theta = I.$$

The realization provides two variants. An orthogonal transform uses $F(x) = Qx$ and $G(y) = Q^\top y$, with construction-time certification $Q^\top Q \approx I$. An affine transform uses $F(x) = Ax + b$ and $G(y) = A^{-1}(y - b)$, with finite values, invertibility, and an induced infinity-norm condition bound.

Definition 4.2 (Transform chain). For an ordered chain $\theta = (\theta_1, \dots, \theta_k)$, define

$$F_\theta = F_{\theta_k} \circ \dots \circ F_{\theta_1}, \quad G_\theta = G_{\theta_1} \circ \dots \circ G_{\theta_k}.$$

The chain cost is $\sum_i \kappa_{\theta_i}$.

Theorem 4.3 (Composition reversibility). *If every transform in θ is exactly admissible, then F_θ is bijective and $G_\theta = F_\theta^{-1}$.*

Proof. Successive cancellation gives

$$G_{\theta_1} \dots G_{\theta_k} F_{\theta_k} \dots F_{\theta_1} = I.$$

The reverse product similarly equals I . Hence the two compositions are mutual inverses. $\square$

Remark 4.4 (Finite precision). The realization does not claim bit-exact machine inversion. Every transform step records its reconstructed input and Euclidean backward error, and the complete chain applies inverse transforms in reverse order and records

$$\varepsilon_{\text{chain}} = \|G_\theta(F_\theta(x)) - x\|_2.$$

The executable contract checks this error relative to $1 + \|x\|_2$.

5 Selector geometry

Definition 5.1 (Selector). A selector is a localized transformed expert

$$s = (id_s, \theta_s, \mu_s, \Theta_s, \mathcal{M}_s, r_s, \nu_s, e_s^0, a_s^0, \ell_s^0, h_s, \text{enabled}_s).$$

Here θ_s is a reversible chart, $\mu_s \in \mathbb{R}^d$ a center, Θ_s a precision parameter, $\mathcal{M}_s$ one candidate model per candidate, $r_s \in [0, 1]$ reliability, $\nu_s \in [0, 1]$ novelty tolerance, $e_s^0 \in [0, 1]$ exploration floor, $a_s^0 \in (0, 1]$ routing floor, $\ell_s^0 \in [a_s^0, 1]$ learning floor, and h_s lifecycle state.

A selector is not the final selection function. It produces a structured vote over candidates.

5.1 Certified precision

Let R be the stored raw lower-triangular parameter and $\gamma > 0$ its factor scale. Define

$$L_{ij} = \begin{cases} 0, & j > i, \\ \sqrt{\gamma}(\text{softplus}(R_{ii}) + 10^{-6}), & j = i, \\ \sqrt{\gamma}R_{ij}, & j < i. \end{cases}$$

Let declared bounds satisfy $0 < \lambda_{\min} < \lambda_{\max}$ and $\kappa_{\max} \geq 1$. Set

$$f = \max\left(\lambda_{\min}, \frac{\lambda_{\max}}{\kappa_{\max}}\right), \quad A = LL^\top.$$

For a real symmetric matrix, define the absolute row-sum bound

$$B(A) = \max_i \sum_j |A_{ij}|.$$

Let

$$\hat{A} = \begin{cases} A, & B(A) \leq \lambda_{\max} - f, \\ \frac{\lambda_{\max} - f}{B(A)}A, & \text{otherwise.} \end{cases} \quad \Lambda = \hat{A} + fI.$$

Theorem 5.2 (Precision spectrum and condition bound). *For every well-formed precision model,*

$$fI \preceq \Lambda \preceq \lambda_{\max}I, \quad \kappa_2(\Lambda) \leq \kappa_{\max}.$$

Proof. $A = LL^\top \succeq 0$. Since A is symmetric, $\lambda_{\max}(A) \leq B(A)$. Construction therefore gives $0 \preceq \hat{A} \preceq (\lambda_{\max} - f)I$. Adding fI bounds every eigenvalue of Λ in $[f, \lambda_{\max}]$. Finally,

$$\kappa_2(\Lambda) \leq \frac{\lambda_{\max}}{f} \leq \kappa_{\max}.$$

□

Corollary 5.3 (Monotone broadening). *For fixed raw factor and bounds, replacing γ by $\alpha\gamma$, $0 < \alpha \leq 1$, cannot increase precision in Loewner order.*

Proof. The realized coefficient on the fixed PSD matrix is the minimum of a nondecreasing linear function of γ and a fixed cap. Reducing γ decreases or preserves the PSD term while leaving fI unchanged. □

5.2 Activation

For input $x \in \mathbb{R}^d$, let

$$z_s = F_{\theta_s}(x), \quad \delta_s = z_s - \mu_s, \quad d_s^2 = \delta_s^\top \Lambda_s \delta_s.$$

The routing activation and learning locality are

$$a_s(x) = \max \left(a_s^0, \exp \left(\max \left(-\frac{1}{2} d_s^2, \log a_s^0 \right) \right) \right), \quad \ell_s(x) = \max(\ell_s^0, a_s(x)).$$

By Theorem 5.2, $d_s^2 \geq 0$, so $a_s, \ell_s \in (0, 1]$.

The deterministic exploration score is

$$e_s(x) = \left[e_s^0 + [\text{inactive}_s / 128]_0^{0.35} + 0.15 \nu_s \right]_0^1.$$

Despite the implementation field name, this is a ranking score rather than a Bernoulli selection probability.

6 Selector votes

Let $\mathcal{C}^+$ be the enabled candidate set. Candidate c has a positive prior propensity $\rho_c \in (0, 1)$. For selector s and candidate c , the additive B-spline score is

$$u_{sc}(z_s) = b_{sc} + \sum_{j=1}^d \sum_k \beta_{scjk} B_{jk}(z_{sj}), \quad \hat{p}_{sc} = \sigma(u_{sc}).$$

Spline axes are finite and nondecreasing, and the closed right endpoint is defined as the final basis vector. Thus the basis partitions unity throughout the declared closed domain.

The local score has eight positive components: candidate prior propensity, selector activation, spline fit, smoothed empirical success, reward quality, freshness, selector reliability, and an uncertainty penalty. If these are $v_{sc1}, \dots, v_{sc8}$, define

$$L_{sc} = \sum_{j=1}^8 \log \text{saf}(v_{scj}), \quad q_{sc} = \frac{\exp(L_{sc})}{\sum_{c' \in \mathcal{C}^+} \exp(L_{sc'})}.$$

q_s is the selector's local operational posterior.

For $|\mathcal{C}^+| > 1$, define normalized entropy

$$H_s = - \frac{\sum_c q_{sc} \log q_{sc}}{\log |\mathcal{C}^+|},$$

and let $H_s = 0$ for one candidate. Define

$$u_s = \left[\frac{1}{2}(1 - a_s) + \frac{1}{2} H_s \right]_0^1,$$

$$\text{support}_{sc} = q_{sc}(1 - u_s), \quad \text{opposition}_{sc} = (1 - q_{sc})(1 - u_s),$$

$$\text{novelty}_s = [(1 - a_s) \nu_s]_0^1, \quad \text{IG}_s = [H_s r_s]_0^1.$$

Definition 6.1 (Selector vote). A selector vote is the immutable tuple containing selector identity, transform and reconstruction traces, activation decomposition, every local candidate decomposition, novelty, entropy-weighted information score, and the reason the selector entered the evaluated set.

Theorem 6.2 (Local simplex). *For every evaluated selector,*

$$q_{sc} \geq 0, \quad \sum_{c \in \mathcal{C}^+} q_{sc} = 1.$$

Proof. Stable softmax subtracts the maximum finite logit, so at least one exponential is one and the denominator is positive. Normalization yields the claim. $\square$

Theorem 6.3 (Vote-mass conservation). *For every selector-candidate pair,*

$$\text{support}_{sc} + \text{opposition}_{sc} + u_s = 1.$$

Moreover,

$$\sum_c \text{support}_{sc} = 1 - u_s.$$

Proof. The first identity follows by factoring $1 - u_s$:

$$q_{sc}(1 - u_s) + (1 - q_{sc})(1 - u_s) + u_s = 1.$$

The second follows from Theorem 6.2. $\square$

7 Bounded exact coalitions

The evaluator first admits the three routable selectors with highest activation, using selector ID as the tie-break. If selectors remain, it adds one selector maximizing

$$e_s + 0.01J_\zeta(id_s),$$

where J_ζ is a deterministic SHA-256-derived jitter in $[0, 1]$ for explicit seed ζ . Thus at most four selectors enter the evaluated vote set V .

Definition 7.1 (Feasible coalition). For budget $M \geq 1$,

$$\mathcal{F}_M(V) = \{K \subseteq V : 1 \leq |K| \leq \min(M, |V|)\}.$$

A coalition is represented by its ascending selector-ID tuple.

For $K \in \mathcal{F}_M(V)$, define

$$\begin{aligned} \text{coverage}(K) &= 1 - \prod_{s \in K} (1 - a_s), \\ \text{diversity}(K) &= \begin{cases} 0, & |K| = 1, \\ \frac{1}{\binom{|K|}{2}} \sum_{\{s,t\} \subseteq K} \frac{1}{2} \sum_c |q_{sc} - q_{tc}|, & |K| > 1, \end{cases} \\ \text{utility}(K) &= \frac{1}{|K|} \sum_{s \in K} [\text{IG}_s + 0.5a_s]_0^1. \end{aligned}$$

For an enabled directed edge internal to K , let

$$w_e = \sigma(1.2 \text{compat}_e + \text{IG}_e - 1.1 \text{redundancy}_e - 1.2 \text{conflict}_e).$$

Redundancy and conflict are the corresponding weighted sums divided by the number of internal enabled directed edges, or zero if none exist. Complexity is the sum of transform-chain costs.

Definition 7.2 (Coalition objective).

$$\mathcal{J}(K) = 1.4 \text{coverage}(K) + 0.8 \text{diversity}(K) + \text{utility}(K) \\ - 0.9 \text{redundancy}(K) - 0.8 \text{conflict}(K) - 0.03 \text{complexity}(K).$$

The selected coalition is

$$K^* = \text{lexmin arg } \max_{K \in \mathcal{F}_M(V)} \mathcal{J}(K).$$

Theorem 7.3 (Exact bounded coalition optimality). ***build_coalition** returns K^* . If several feasible coalitions have maximum score, it returns the lexicographically smallest selector tuple.*

Proof. Identifier-sorted votes establish a canonical order. Enumeration by **combinations** visits every nonempty subset of sizes $1, \dots, \min(M, |V|)$ exactly once. Minimizing $(-\mathcal{J}(K), \text{ids}(K))$ first maximizes the score and then applies the stated tie-break. $\square$

Corollary 7.4 (Bounded search). *Under the default preselection bound, at most*

$$\sum_{r=1}^4 \binom{4}{r} = 15$$

coalitions are scored.

This is exact optimality over the evaluated vote set, not over every selector in the registry.

8 Coalition posterior

Normalize enabled prior propensities:

$$\pi_c = \frac{\rho_c}{\sum_{j \in \mathcal{C}^+} \rho_j}.$$

For $s \in K^*$, let

$$\omega_s = a_s r_s, \quad Z_\omega = \max \left(1, \sum_{s \in K^*} \omega_s \right), \quad w_s = \frac{\omega_s}{Z_\omega}.$$

Then $w_s \geq 0$ and $W = \sum_s w_s \leq 1$. With $\bar{\pi}_c = \text{saf}(\pi_c)$ and $\bar{q}_{sc} = [q_{sc}]_\epsilon^{1-\epsilon}$, define

$$G_c = \log \bar{\pi}_c + \sum_{s \in K^*} w_s (\log \bar{q}_{sc} - \log \bar{\pi}_c),$$

$$P(c \mid x, X) = \frac{\exp(G_c)}{\sum_{j \in \mathcal{C}^+} \exp(G_j)}.$$

The winner is the lexicographically smallest candidate ID among posterior maximizers.

Theorem 8.1 (Tempered log-pool identity).

$$P(c \mid x, X) \propto \bar{\pi}_c^{1-W} \prod_{s \in K^*} \bar{q}_{sc}^{w_s}.$$

The prior has nonnegative residual weight $1 - W$.

Proof. Expanding G_c gives

$$G_c = (1 - W) \log \bar{\pi}_c + \sum_s w_s \log \bar{q}_{sc}.$$

Exponentiation and normalization yield the result. $\square$

Corollary 8.2 (Prior neutrality). *If every selected local report equals π , then $P = \pi$.*

Proof. Substitute $\bar{q}_{sc} = \bar{\pi}_c$ into G_c . Every evidence difference is zero, so softmax normalizes $\log \pi_c$ back to π_c . $\square$

Theorem 8.3 (Coalition simplex).

$$P(c \mid x, X) \geq 0, \quad \sum_{c \in \mathcal{C}^+} P(c \mid x, X) = 1.$$

Proof. The proof is identical to Theorem 6.2. $\square$

9 Posterior alignment

Posterior alignment is a diagnostic certificate, not another posterior. Let c^* be the selected candidate. For each $s \in K^*$, define

$$x_s = q_{s,c^*}, \quad \omega_s = a_s \tilde{r}_s, \quad \Omega = \max \left(\sum_s \omega_s, 10^{-12} \right),$$

where $\tilde{r}_s$ is the reliability used in the local component. Define weighted mean and variance

$$\mu = \frac{\sum_s \omega_s x_s}{\Omega}, \quad \sigma^2 = \frac{\sum_s \omega_s (x_s - \mu)^2}{\Omega}.$$

Then

$$\text{agreement} = \left[\mu(1 - \sqrt{\max(0, \sigma^2)}) \right]_0^1,$$

$$\text{independent} = [\mu(1 - \text{redundancy}(K^*))]_0^1.$$

Weighted dissent, uncertainty, and novelty masses are

$$d = \frac{\sum_s \omega_s \text{opposition}_{s,c^*}}{\Omega}, \quad u = \frac{\sum_s \omega_s u_s}{\Omega}, \quad n = \frac{\sum_s \omega_s \text{novelty}_s}{\Omega}.$$

Definition 9.1 (Posterior alignment). The alignment certificate is

$$\mathcal{A} = (P(c^*), \text{agreement}, \text{independent}, \text{redundancy}, d, u, n, A),$$

where

$$A = \left[0.35P(c^*) + 0.30\text{agreement} + 0.25\text{independent} \right. \\ \left. - 0.15\text{redundancy} - 0.15d - 0.10u - 0.10n \right]_0^1.$$

Proposition 9.2 (Alignment range). *Every component of $\mathcal{A}$ lies in $[0, 1]$.*

Proof. All local probabilities, weights, and masses lie in $[0, 1]$. Their normalized weighted means remain in $[0, 1]$. Since $x_s \in [0, 1]$, $\sigma^2 \leq 1/4$; the agreement expression is bounded and then clipped. Independent support and the final index are explicitly clipped. $\square$

10 Structural proposals

Definition 10.1 (Structural proposal). A structural proposal is the immutable advisory tuple

$$\mathcal{P} = (op, I, E, \Phi, R, \hat{g}, c, \text{advisory}),$$

where:

- op belongs to the following closed operation enum:
 - `broaden_selector`;
 - `create_specialist_selector`;
 - `disconnect_redundant_selectors`; or
 - `shadow_alternative_coalition`;
- I is a nonempty canonical selector-ID tuple;
- E is a finite numeric evidence tuple;
- Φ is a finite set of stated preconditions;
- R is human-readable rationale;
- $\hat{g} \in [0, 1]$ is an estimated, not proven, gain;
- $c \geq 0$ is a complexity estimate; and
- `advisory` = `true`.

The fixed proposal policy emits:

1. `broaden_selector` when an evaluated routing activation is below 10^{-3} ;
2. `create_specialist_selector` when weighted novelty mass exceeds 0.55;
3. `disconnect_redundant_selectors` when coalition redundancy exceeds 0.65; and
4. `shadow_alternative_coalition` when weighted dissent exceeds 0.55.

Theorem 10.2 (Proposal non-interference). *Evaluation and proposal emission do not alter X , and no structural proposal can change the selected candidate in the decision that contains it.*

Proof. The evaluator receives an immutable snapshot and constructs new frozen records. Proposal construction has no reference to a state-publication primitive. The selected candidate is computed before proposal emission and stored in the same immutable decision. $\square$

No theorem of structural improvement or structural rollback is claimed. Those require typed application functions, checked postconditions, retained parent snapshots, and realized-gain evaluation.

11 Evaluation relation and deterministic replay

Definition 11.1 (Evaluation relation). For explicit time $t \in \mathbb{N}$, seed $\zeta \in \mathbb{Z}$, and coalition budget $M \geq 1$, write

$$X, x, t, \zeta, M \vdash D$$

when D is constructed by:

1. validating finite input of dimension d ;
2. computing geometry for every routable selector;

3. selecting the top three activations and one deterministic exploratory selector if available;
4. evaluating selected votes in selector-ID order;
5. computing the exact bounded coalition;
6. aggregating the tempered log-pool posterior;
7. applying canonical candidate tie-breaking;
8. computing alignment, novelty, and proposals; and
9. recording $(\text{CF-1XQ-1.0}, h(X), g, t, \zeta)$ in D .

When evaluation time is omitted, the realization samples the monotonic clock once and records it. A missing seed resolves to zero.

Theorem 11.2 (Functional evaluation). *For fixed (X, x, t, ζ, M) , the relation $X, x, t, \zeta, M \vdash D$ has exactly one result.*

Proof. All state registries and local reports are canonicalized. Activation ties use selector IDs; exploratory jitter is a deterministic hash of seed and selector ID; coalition ties use ascending selector tuples; and candidate ties use candidate IDs. Every downstream operation is deterministic. $\square$

Remark 11.3. Bit-for-bit equality across Python builds or hardware platforms additionally depends on IEEE-754 and `libm` behavior. The theorem concerns the defined realization environment or exact-real model.

Proposition 11.4 (Registry permutation invariance). *Permuting transform, candidate, selector, edge, or selector-candidate registry inputs without changing semantic fields yields the same canonical snapshot and the same decision for fixed evaluation arguments.*

Proof. Construction sorts non-semantic registries by their unique identifiers. Semantic sequences, notably transform-chain and spline-axis order, are preserved. $\square$

12 Outcomes and generation transitions

Definition 12.1 (Outcome). An outcome is

$$o = (g, x, c, r, b, \lambda, t_o, D, m, h),$$

where $r \in [0, 1]$ is reward, b success, $\lambda \geq 0$ latency, t_o observed time, D the originating decision, m the model identifier, and h the snapshot fingerprint.

Definition 12.2 (Outcome validity). $\text{Valid}_X(o)$ holds exactly when:

1. $o.g = X.g$, $o.m = \text{CF-1XQ-1.0}$, and $o.h = h(X)$;
2. the embedded decision has the same model, generation, and fingerprint; and
3. the redundant input and selected-candidate fields equal those in the embedded decision.

This is content binding, not cryptographic authentication. A malicious caller able to forge complete in-memory objects is outside the current trust model.

Definition 12.3 (Pure update). For a queue-ordered outcome sequence O , let O_X be its valid subsequence. The pure update is

$$\mathcal{U}(X, O, t_c) = \begin{cases} X, & O_X = \emptyset, \\ \text{Learn}(X, O_X, t_c), & O_X \neq \emptyset. \end{cases}$$

For a nonempty valid batch, the successor has generation $g + 1$. Candidate definitions, transforms, and topology are unchanged; selector models, health, precision, and edge statistics may change.

The learning fold is intentionally queue-order-sensitive. Batch permutation invariance is not claimed.

Theorem 12.4 (Generation monotonicity and old-state preservation).

$$\text{generation}(\mathcal{U}(X, O, t_c)) - \text{generation}(X) \in \{0, 1\}.$$

The difference is one exactly when O_X is nonempty. In either case, the original snapshot X is unchanged.

Proof. An empty valid set returns the same object. Otherwise the update constructs a new frozen snapshot with generation $g + 1$ by functional replacement. No operation mutates a field of X . $\square$

Theorem 12.5 (Direct foreign-evidence exclusion). *An outcome from a distinct same-numbered snapshot branch is not included in the valid learning subsequence O_X unless the two canonical identities collide. Equivalently, no such occurrence is directly applied as learning evidence.*

Proof. $\text{Valid}_X(o)$ requires both generation and fingerprint equality. A different canonical state has a different mathematical identity. The implementation conclusion is conditioned on SHA-256 collision resistance. $\square$

This is an application-exclusion property, not full non-interference. Because the queue is bounded, a foreign outcome can occupy capacity or evict an older local outcome before publication and thereby influence later evolution indirectly. Preventing that channel would require admission-time validation or separate capacity.

Proposition 12.6 (Update range safety). *For a well-formed snapshot, authentic outcomes, and learning rates in $[0, 1]$: observation and success counts remain valid; reward means stay in $[0, 1]$; latency means remain nonnegative; selector reliability remains in $[0.01, 1]$; edge statistics remain in $[0, 1]$; and updated precision satisfies Theorem 5.2.*

Proof. Counts increment by nonnegative integers and success increments by zero or one. Running means are convex combinations. Reliability and edge updates are clipped. Precision adaptation forms a convex combination in the PSD cone, refactorizes by Cholesky, and re-enters the certified precision constructor. $\square$

13 Concurrent snapshot publication

The reference register has two locks: a publication lock and an outcome lock. A resolution holds the publication lock only long enough to capture one snapshot reference. Evaluation then proceeds outside the lock against the captured immutable value. Observation appends under the outcome lock. A publisher holds the publication lock, then the outcome lock, while it captures the current batch, validates and learns, clears the batch only after successful successor construction, and swaps the snapshot reference.

Theorem 13.1 (Per-resolution snapshot consistency). *Every completed resolution is derived from exactly one snapshot X . Its decision satisfies*

$$D.\text{generation} = X.g, \quad D.\text{snapshot_fingerprint} = h(X).$$

A concurrent publication cannot cause one decision to mix fields from two generations.

Proof. **resolve** captures one snapshot reference under the publication lock and passes only that reference into evaluation. Valid snapshots are transitively composed of canonical immutable values. Publication constructs a different snapshot and replaces the register reference. It does not mutate the captured value. The TLAPS predicate **ResolutionSnapshotConsistency** machine-checks the generation and fingerprint metadata portion of this claim. The stronger field-level no-mixing conclusion additionally uses the Python immutable-object capture argument and is not represented field by field in the TLA+ state. $\square$

Theorem 13.2 (Restricted-history implementation linearization points). *For completed in-process calls among the listed API operations, under the stated lock semantics:*

- ***resolve** linearizes at its snapshot-reference capture;*
- ***observe** linearizes at its locked queue append, including any same-critical-section eviction and drop-count increment;*
- *an empty **publish_generation** linearizes at the locked empty-queue check; and*
- *a successful nonempty **publish_generation** linearizes at the snapshot-reference assignment after queue clearing and rejection accounting.*

Proof. Publishers cannot overlap. While publication holds its lock, no resolver can capture the registry reference. The outcome lock prevents an append from entering the captured batch after its cut. Thus an overlapping observation is ordered entirely before or after publication. From queue capture through the snapshot assignment, the publisher holds both locks, so no public register or counter operation can observe the intermediate clear, count update, or reference swap. A resolution therefore sees the complete old or complete new snapshot. The queue copy fixes the batch; it is not itself the successful nonempty linearization point. $\square$

Theorem 13.3 (In-process rollback on successor-construction failure). *If successor construction raises synchronously before the commit section, then after stack unwinding the captured outcome queue, current snapshot, and published drop/rejection counters remain unchanged.*

Proof. The queue is held under the outcome lock and is not cleared until **apply_outcomes** has returned a successor. The snapshot reference is assigned and the rejection counter updated only after that return. Context manager exit releases both locks. This theorem does not cover process crash, power loss, asynchronous interruption, durable recovery, or an arbitrary fault after the queue-clear commit section begins. $\square$

A failed publication call has no successful publication linearization point; it stutters on the externally visible abstract state. In the TLA+ model it may change only publisher-local control and a ghost failure marker.

The queue has finite capacity. When full, the oldest outcome is deliberately discarded and **dropped_outcomes** increments. Stale or fingerprint-mismatched entries are deliberately removed during a successful publication attempt and **rejected_outcomes** increments. The architecture therefore proves bounded storage and explicit accounting, not lossless evidence retention. The full conservation equation uses ghost sets for queued, dropped, rejected, and applied occurrences in the TLA+ module; the Python object exposes only drop and rejection counters.

14 Mechanically checked publication safety

The module `CognitiveFabricPublication.tla` makes the concurrency boundary executable as a transition system. Its parameter assumptions are R finite with $R \neq \emptyset$, $C \in \mathbb{N}_{>0}$, and $N \in \mathbb{N}_{>0}$ for the resolver set, queue capacity, and bound on distinct outcome occurrences. Publisher control is deliberately exposed as

$$\text{idle} \rightarrow \text{publish_locked} \rightarrow \text{building} \rightarrow \text{ready} \rightarrow \text{idle}.$$

The action relation contains resolver capture and completion, local, foreign, and malformed observation, the four publication phases, the empty path, and the exception path. The behavior specification is

$$\text{SafetySpec} \equiv \text{Init} \wedge \Box[\text{Next}]_{\text{vars}}.$$

The history function `outcomeById`, disposition sets, commit log, and application counter are ghost state: they make occurrence identity and accounting explicit without requiring the Python realization to store them. In this module, a local `Observe` consumes one resolver's `ready` token, so the direct trace correspondence assumes at most one local observation per modeled resolution completion. The Python API does not enforce that client discipline and accepts repeated use of a decision. That difference is another explicit reason the present traceability map is not an implementation refinement theorem.

The checked safety predicate is the conjunction

$$\begin{aligned} \text{Safety} \equiv & \text{TypeOK} \wedge \text{BoundedQueue} \wedge \text{UniqueOutcomeIds} \wedge \text{DispositionDisjoint} \\ & \wedge \text{OutcomeAccounting} \wedge \text{CountAccuracy} \wedge \text{SnapshotGenerationAgreement} \\ & \wedge \text{PublisherLockInvariant} \wedge \text{ResolutionSnapshotConsistency} \\ & \wedge \text{CorrectCommitClassification} \wedge \text{ForeignApplicationExclusion}. \end{aligned}$$

In particular, every created occurrence belongs to exactly one of the current queue, dropped, rejected, or applied sets; counters equal the corresponding cardinalities; every commit partitions its cut into exactly the identifiers valid and invalid at the captured base; and no foreign occurrence enters the applied set. The inductive strengthening records occurrence identity and the source shape needed to derive direct foreign-application exclusion.

Theorem 14.1 (Mechanized publication invariant). *Under `ParameterAssumptions`,*

$$\text{SafetySpec} \implies \Box \text{Safety}.$$

Proof. The accompanying proof module shows that `Init` establishes `InductiveSafety`; proves preservation for every one of the eleven `Next` branches; proves preservation by the stuttering branch in $[\text{Next}]_{\text{vars}}$; and applies temporal induction. TLAPS rolling 1.6.0-pre build 3ab43c7, invoked with `-strict -cleanfp -C`, discharged all 747 obligations. Its final Isabelle reconstruction reported no errors [22]. The module also proves the failure action preserves the evidence state and that a successful commit increments the generation exactly when its valid set is nonempty. $\square$

This is a symbolic theorem over every model instance satisfying the parameter assumptions, rather than a finite enumeration; N is nevertheless an explicit per-instance bound on distinct occurrence identifiers. It proves the abstract safety predicate, not a statement-level Python refinement or a general atomic-object linearizability theorem. As an independent finite cross-check, TLC 2026.03.02.213938 used $|R| = 2$, $C = 2$, and $N = 4$. It explored 920,549 generated states, 402,582 distinct states, and depth 38 without error, while also checking `PublicationEventuallyReturns` under weak fairness for an already-active publisher. That

liveness check is finite-instance evidence; the TLAPS theorem above is a safety result. Deadlock checking is disabled in the TLC configuration. The fairness condition begins only after a publisher has acquired its lock; it asserts neither eventual client requests, lock acquisition fairness, nor Python scheduler progress.

15 Reference realization and traceability map

The Python module `cognitive_fabric_reference.py` is one standard-library finite-precision realization. Table 1 maps formal objects to implementation symbols for auditability. The table is a traceability relation, not a mechanically proved refinement mapping.

Formal object or property	Reference symbol
Certified coordinate maps	<code>OrthogonalTransform</code> , <code>AffineTransform</code>
Chain inverse and backward certificate	<code>apply_transform_chain</code> , <code>TransformChainTrace</code>
Certified precision	<code>PrecisionModel.matrix</code> , <code>symmetric_spectral_upper_bound</code>
Selector	<code>SelectorRegion</code>
Selector vote and q_{sc}	<code>evaluate_selector</code> , <code>SelectorVote</code> , <code>CandidateScoreDecomposition.local_posterior</code>
Coalition objective and exact optimum	<code>score_coalition</code> , <code>build_coalition</code>
Tempered log pool	<code>aggregate_coalition</code>
Alignment certificate	<code>posterior_alignment</code> , <code>PosteriorAlignment</code>
Typed advisory proposal	<code>StructuralOperation</code> , <code>StructuralProposal</code>
Snapshot and well-formedness	<code>CognitiveFabricSnapshot</code> , <code>validate_snapshot</code>
Canonical content identity	<code>snapshot_fingerprint</code>
Evaluation relation	<code>evaluate_snapshot</code>
Executable decision checks	<code>decision_invariant_violations</code> , <code>assert_decision_invariants</code>
Outcome validity and pure update	<code>outcome_matches_snapshot</code> , <code>apply_outcomes</code>
Concurrent register	<code>CognitiveFabric.resolve</code> , <code>observe</code> , <code>publish_generation</code>

Table 1: Formal-to-code traceability map.

15.1 Exact and computational assumptions

The realization and correspondence claim is conditional on:

1. exact-real algebra for the mathematical theorems;
2. tolerance-bounded orthogonality and backward error for IEEE-754 realization;
3. finite inputs and validated constructors;
4. the lock semantics of the supported Python runtime;
5. at most one local observation per modeled resolution completion when comparing Python publication traces with the TLA+ module;
6. deterministic behavior within a fixed runtime and math library; and
7. SHA-256 collision resistance for direct computational foreign-evidence exclusion.

The Jacobi eigensolver in the code is diagnostic. The precision theorem does not rely on it; it relies on the certified absolute row-sum bound.

16 Executable conformance evidence

The accompanying `unittest` suite exercises the following conformance properties:

- rejection of false orthogonality and ill-conditioned affine maps;
- complete reverse-order chain reconstruction;
- mutation isolation for caller-owned matrices;
- certified precision floor, ceiling, and condition bounds over generated matrices;
- monotone broadening in Loewner order;
- B-spline nonnegativity and partition of unity on the closed domain;
- canonical snapshot identity under registry permutations;
- rejection of duplicate, missing, unknown, and non-finite state;
- deterministic replay for fixed snapshot, time, seed, and input;
- local and coalition simplex invariants and vote-mass conservation;
- exact coalition agreement with independent feasible-subset enumeration;
- prior neutrality of the tempered log pool;
- consistent exclusion of disabled candidates;
- advisory proposal non-interference;
- cumulative repeated-outcome spline learning and nontrivial precision learning;
- lifecycle reachability of probation;
- old-snapshot preservation and generation increment;
- occurrence-level application of the captured queue in a two-publisher in-process schedule, without claiming durable exactly-once delivery;
- evidence retention after an injected synchronous successor-construction exception;
- explicit stale and foreign-fingerprint rejection accounting; and
- an in-flight resolution that completes against its captured old snapshot after a concurrent successor is published.

These tests are executable witnesses of conformance. They neither substitute for the abstract TLAPS theorem nor establish the missing Python refinement proof. QuickCheck established the modern property-based testing pattern [20]; Alloy and Dafny remain natural tools for structural-state exploration and implementation verification respectively [23, 24].

17 Related work and novelty boundary

Mixtures and local experts. Adaptive and hierarchical mixtures of local experts introduced learned gating over specialized predictors [1, 2]. Sparse mixture-of-experts systems scale this idea to conditional computation [3]. The Cognitive Fabric selector is best understood as an interpretable, ellipsoidally gated local expert with a reversible coordinate chart. Its novelty is not the existence of gating.

Probability aggregation. The combination of expert probability distributions, including logarithmic pooling, has a substantial literature [4]. Products of experts and committee machines similarly multiply or combine expert evidence [5, 6]. The contribution here is the explicit prior-tempered identity in Theorem 8.1, its trace decomposition, and its integration with bounded exact coalition selection. Empirical calibration remains to be demonstrated.

Coalition and ensemble selection. Dynamic ensemble and classifier selection choose subsets based on local competence [7, 8]. Coalition-structure generation and submodular optimization provide broader combinatorial frameworks [9, 10]. Here the evaluated family is deliberately bounded, so exhaustive enumeration gives an exact theorem rather than an approximation claim.

Reversible representations. Normalizing flows use learned invertible transforms for tractable density models [11]. The Cognitive Fabric does not claim to be a normalizing flow: it contains no Jacobian likelihood objective. Reversibility is an architectural coordinate invariant.

Immutable state and concurrency. Versioned objects, multi-version concurrency, and read-copy-update motivate readers operating on immutable versions while writers publish successors [12, 13, 14]. The contribution here is the formal application to an adaptive decision registry and the binding of every decision and outcome to canonical source state, not a new general concurrency primitive.

Structural adaptation. Systems such as Cascade-Correlation and AdaNet learn or grow model structure [15, 16]. This realization is intentionally weaker: it emits typed proposals but does not apply them. Any future claim of self-improving structure must measure realized utility, check successor well-formedness, and provide rollback.

Delayed feedback. Only the selected candidate receives reward feedback, placing empirical evaluation near contextual-bandit settings [17, 18]. Regret, calibration, and drift performance are not established by the architectural theorems.

18 Limitations and proof roadmap

The present paper is strong enough to serve as a mathematical architecture specification and arXiv systems/formalism preprint. A competitive archival submission should add:

1. a statement-level refinement from the Python lock protocol to an atomic reference specification, with explicit call/return histories for a mechanically checked linearizability theorem;
2. a mechanized algebraic development in Lean, Dafny, or a comparable system for precision bounds, pooling, and update range preservation;
3. contextual-bandit and drift experiments against global predictors, top- k mixture routers, dynamic ensemble selection, and LinUCB-style baselines;
4. calibration experiments using negative log likelihood, Brier score, expected calibration error, and risk-coverage analysis [19];
5. exact-search versus heuristic coalition ablations and edge-objective sensitivity;
6. contention stress tests with occurrence-ID instrumentation reconciling created, queued, applied, dropped, and rejected outcomes; and

7. a separate structural transaction layer with typed operation parameters, checked preconditions, successor validation, parent retention, inverse or rollback certificates, and realized-gain accounting.

The following claims remain explicitly outside the current theorem set: machine-exact transform inversion; cryptographic authenticity; injectivity of SHA-256; calibrated Bayesian inference; mutual-information semantics for the information score; batch permutation invariance; lossless bounded evidence; global coalition optimality over the full registry; executable structural improvement; reversible learning; crash consistency; durable exactly-once delivery; scheduler fairness; symbolic liveness; generic full-API linearizability; and an implementation refinement theorem.

19 Conclusion

The Cognitive Fabric is now specified as more than a software release. A selector is a validated local expert in a reversible coordinate chart. A coalition is the exact maximizer of a stated objective over a bounded feasible family. The candidate posterior is a prior-tempered logarithmic opinion pool. Posterior alignment is an eight-part diagnostic certificate. A structural proposal is a typed advisory value with evidence and preconditions, not an implicit mutation. Evaluation is a functional relation for explicit context, and publication constructs immutable, fingerprinted generations with a restricted-history implementation linearization-point argument and in-process evidence rollback. Its publication safety core is a TLA+ transition system whose inductive invariant and temporal lifting are checked by TLAPS, with bounded TLC evidence as a separate cross-check.

This separation makes the architecture falsifiable. Every theorem names its assumptions; each listed implementation object has an explicit formal correspondence; and every non-theorem is stated as future work rather than hidden inside a name.

A Reference constants

Quantity	Symbol	Reference value
Safe probability floor	ϵ	10^{-12}
Orthogonality tolerance	ε_Q	10^{-10}
Precision diagonal offset	ε_L	10^{-6}
Top activation admissions	k	3
Maximum evaluated selectors	$ V $	4
Default coalition budget	M	4
Exploration jitter scale		0.01
Default outcome capacity		4096
Formal model identifier		CF-1XQ-1.0

B Canonical decision obligations

For a returned decision D , the executable checker requires:

1. model identifier equality and nonnegative evaluation time;
2. novelty and alignment indices in $[0, 1]$;
3. a nonempty finite coalition posterior on the simplex;

4. canonical candidate argmax;
5. a nonempty, duplicate-free coalition contained in the vote set;
6. every local posterior on the simplex;
7. chain reconstruction error within tolerance; and
8. support plus opposition plus uncertainty equal to one for every local candidate record.

C Reproduction commands

```
python3 -m unittest -v test_cognitive_fabric_reference.py
python3 cognitive_fabric_reference.py

# TLAPS rolling 1.6.0-pre, build 3ab43c7
tlapm --strict --cleanfp -C \
  formal/CognitiveFabricPublicationProof.tla

# TLA Tools 2026.03.02.213938
cd formal
java -cp "$TLA2TOOLS" tlc2.TLC -workers 1 \
  -config CognitiveFabricPublication.cfg \
  CognitiveFabricPublication.tla
cd ..
```

For a separate SANY parse, set `TLAPS_STDLIB` to the TLAPS standard module directory and run:

```
java -DTLA-Library="$TLAPS_STDLIB" -cp "$TLA2TOOLS" \
  tla2sany.SANY formal/CognitiveFabricPublicationProof.tla
```

The recorded clean result is 747 TLAPS obligations followed by an error-free Isabelle reconstruction. The recorded TLC result is 920,549 generated states, 402,582 distinct states, depth 38, and no error.

References

- [1] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. <https://doi.org/10.1162/neco.1991.3.1.79>.
- [2] M. I. Jordan and R. A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2):181–214, 1994. <https://doi.org/10.1162/neco.1994.6.2.181>.
- [3] N. Shazeer et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv:1701.06538, 2017. <https://arxiv.org/abs/1701.06538>.
- [4] C. Genest and J. V. Zidek. Combining probability distributions: A critique and an annotated bibliography. *Statistical Science*, 1(1):114–135, 1986. <https://doi.org/10.1214/ss/1177013825>.
- [5] G. E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771–1800, 2002. <https://doi.org/10.1162/089976602760128018>.

- [6] V. Tresp. A Bayesian committee machine. *Neural Computation*, 12(11):2719–2741, 2000. <https://doi.org/10.1162/089976600300014908>.
- [7] K. Woods, W. P. Kegelmeyer, Jr., and K. Bowyer. Combination of multiple classifiers using local accuracy estimates. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(4):405–410, 1997. <https://doi.org/10.1109/34.588027>.
- [8] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes. Ensemble selection from libraries of models. In *Proceedings of ICML*, 2004. <https://doi.org/10.1145/1015330.1015432>.
- [9] T. Sandholm, K. Larson, M. Andersson, O. Shehory, and F. Tohmé. Coalition structure generation with worst case guarantees. *Artificial Intelligence*, 111(1–2):209–238, 1999. [https://doi.org/10.1016/S0004-3702\(99\)00036-3](https://doi.org/10.1016/S0004-3702(99)00036-3).
- [10] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14:265–294, 1978. <https://doi.org/10.1007/BF01588971>.
- [11] L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density estimation using Real NVP. arXiv:1605.08803, 2016. <https://arxiv.org/abs/1605.08803>.
- [12] D. P. Reed. *Naming and Synchronization in a Decentralized Computer System*. PhD thesis, Massachusetts Institute of Technology, 1978. <https://dspace.mit.edu/handle/1721.1/16279>.
- [13] P. A. Bernstein and N. Goodman. Multiversion concurrency control—theory and algorithms. *ACM Transactions on Database Systems*, 8(4):465–483, 1983. <https://doi.org/10.1145/319996.319998>.
- [14] P. E. McKenney and J. D. Slingwine. Read-copy update: Using execution history to solve concurrency problems. In *Parallel and Distributed Computing and Systems*, 1998. <https://www.rdrop.com/users/paulmck/RCU/rclockpdcproof.pdf>.
- [15] S. E. Fahlman and C. Lebiere. The Cascade-Correlation learning architecture. In *Advances in Neural Information Processing Systems 2*, 1989. <https://proceedings.neurips.cc/paper/1989/hash/69adc1e107f7f7d035d7baf04342e1ca-Abstract.html>.
- [16] C. Cortes, X. Gonzalvo, V. Kuznetsov, M. Mohri, and S. Yang. AdaNet: Adaptive structural learning of artificial neural networks. In *Proceedings of ICML*, PMLR 70:874–883, 2017. <https://proceedings.mlr.press/v70/cortes17a.html>.
- [17] L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of WWW*, 2010. <https://doi.org/10.1145/1772690.1772758>.
- [18] W. Chu, L. Li, L. Reyzin, and R. Schapire. Contextual bandits with linear payoff functions. In *Proceedings of AISTATS*, PMLR 15:208–214, 2011. <https://proceedings.mlr.press/v15/chu11a.html>.
- [19] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *Proceedings of ICML*, PMLR 70:1321–1330, 2017. <https://proceedings.mlr.press/v70/guo17a.html>.
- [20] K. Claessen and J. Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of ICFP*, 2000. <https://doi.org/10.1145/351240.351266>.

- [21] L. Lamport. The temporal logic of actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994. <https://doi.org/10.1145/177492.177726>.
- [22] D. Cousineau, D. Doligez, L. Lamport, S. Merz, D. Ricketts, and H. Vanzetto. TLA+ proofs. arXiv:1208.5933, 2012. <https://arxiv.org/abs/1208.5933>.
- [23] D. Jackson. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology*, 11(2):256–290, 2002. <https://doi.org/10.1145/505145.505149>.
- [24] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR-16*, LNCS 6355:348–370, 2010. https://doi.org/10.1007/978-3-642-17511-4_20.